

APOSTILA COMPLETA

Java Arrays

Do básico aos arrays multidimensionais

Declaração • Inicialização • For-each • java.util.Arrays • Busca Binária • Varargs • 2D Arrays • Desafios

12 capítulos • Exemplos práticos • 3 desafios

Sumário

01	Introdução aos Arrays	O que são, por que usar e a motivação
02	Declarando e Inicializando Arrays	Sintaxe, valores padrão e .length
03	Arrays como Objetos em Java	Herança, referências e heap
04	O Enhanced For Loop (For-each)	Sintaxe, uso correto e limitações
05	A Classe java.util.Arrays	sort, fill, copyOf e toString
06	Busca em Arrays	Busca linear vs busca binária
07	Variable Arguments (Varargs)	Sintaxe ... e regras de uso
08	Arrays Bidimensionais	Matrizes, índices e jagged arrays
09	Arrays Multidimensionais	Além do 2D, casos de uso
10	Desafio: Elemento Mínimo	readIntegers e findMin
11	Desafio: Inversão de Array	Troca in-place até o meio
12	Desafio: Ordenação Decrescente	Comparator.reverseOrder()

CAPÍTULO 01

Introdução aos Arrays

O que são arrays, por que usá-los e a motivação com o problema prático

Por que precisamos de Arrays?

Imagine que você está desenvolvendo um sistema de pedidos para uma hamburgueria. Você precisa armazenar os preços de 10 hambúrgueres diferentes. Sem arrays, seria necessário declarar 10 variáveis separadas: `preco1`, `preco2`, `preco3...` — uma abordagem trabalhosa, repetitiva e completamente impraticável quando o número de itens cresce para centenas ou milhares.

Arrays resolvem esse problema de forma elegante: em vez de criar N variáveis, você cria **uma única estrutura** capaz de armazenar N valores do mesmo tipo. Isso torna o código mais organizado, mais legível e muito mais fácil de manter. Em Java, arrays são a estrutura de dados mais fundamental — todos os programas, desde os mais simples até os mais complexos, fazem uso de arrays em algum momento.

O que é um Array?

Um array é uma **estrutura de dados linear** que armazena uma sequência de valores do mesmo tipo, organizados de forma contígua na memória. Em Java, você pode criar arrays de qualquer tipo primitivo (como `int`, `double`, `boolean`, `char`) ou de qualquer classe (`String`, `Object`, etc.).

Cada elemento dentro do array é identificado por um **índice numérico**, que começa sempre em **0** (zero). Portanto, em um array com 5 elementos, os índices válidos são 0, 1, 2, 3 e 4. Tentar acessar um índice fora desse intervalo — por exemplo, o índice 5 ou um valor negativo — causa uma exceção em tempo de execução chamada `ArrayIndexOutOfBoundsException`, um dos erros mais comuns entre iniciantes em Java.

```
int[] numbers = new int[5]; // declara e aloca array de 5 inteiros

int[] primes = {2, 3, 5, 7, 11}; // array initializer – valores já definidos

System.out.println(primes[0]); // imprime: 2 (primeiro elemento)
```

```
System.out.println(primes[4]); // imprime: 11 (último elemento)

System.out.println(primes.length); // imprime: 5 (tamanho do array)

// primes[5] lançaria ArrayIndexOutOfBoundsException!
```

■ **Dica:** A propriedade `length` retorna o número total de elementos do array. Lembre-se: o último índice válido é sempre `length - 1`, não `length`.

CAPÍTULO 02

Declarando e Inicializando Arrays

Sintaxe de declaração, alocação com `new`, valores padrão e a propriedade `length`

Formas de Declarar um Array

Em Java, existem duas formas principais de declarar e inicializar um array. A primeira utiliza a palavra-chave `new` para alocar espaço na memória, definindo apenas o tamanho. Nesse caso, todos os elementos recebem automaticamente **valores padrão** de acordo com o tipo: `0` para inteiros, `0.0` para ponto flutuante, `false` para booleanos e `null` para referências a objetos.

A segunda forma utiliza o **array initializer**, que consiste em fornecer os valores entre chaves `{ }` diretamente na declaração. Nesse caso, o tamanho do array é determinado automaticamente pela quantidade de valores fornecidos, e não é necessário usar a palavra-chave `new` explicitamente.

```
// Forma 1: new — aloca com valores padrão

int[] nums = new int[5]; // [0, 0, 0, 0, 0]

String[] names = new String[3]; // [null, null, null]

boolean[] flags = new boolean[4]; // [false, false, false, false]

double[] prices = new double[2]; // [0.0, 0.0]


// Forma 2: array initializer — valores já definidos

int[] primes = {2, 3, 5, 7, 11};

String[] fruits = {"Apple", "Banana", "Cherry"};

double[] rates = {1.5, 2.0, 3.75};


// Array 2D com initializer aninhado
```

```
int[][] matrix = {{1,2,3},{4,5,6},{7,8,9}};
```

A Propriedade length

Todo array em Java possui a propriedade `length`, que retorna o número total de elementos contidos nele. É importante notar que `length` é uma **propriedade** (campo), não um método — portanto não se usa parênteses ao acessá-la. Isso contrasta com `String.length()`, que é um método e requer parênteses. Essa é uma fonte comum de confusão para iniciantes.

```
int[] arr = {10, 20, 30, 40, 50};

System.out.println(arr.length); // correto: 5

// System.out.println(arr.length()); // ERRO de compilação!

// Uso clássico com for loop:

for (int i = 0; i < arr.length; i++) {

    System.out.println("arr[" + i + "] = " + arr[i]);

}
```

■ **Dica:** Nunca hardcode o tamanho do array dentro de loops. Sempre use `arr.length` — assim, se o array mudar de tamanho, o loop continuará funcionando corretamente.

CAPÍTULO 03

Arrays como Objetos em Java

Herança de Object, semântica de referências, heap e como copiar corretamente

Arrays são Objetos Especiais

Em Java, **tudo é um objeto** — e arrays não são exceção. Um array é uma classe especial que herda diretamente de `java.lang.Object`. Isso significa que arrays têm acesso a métodos como `toString()`, `equals()` e `hashCode()` — embora as implementações padrão nem sempre façam o que você esperaria (por exemplo, `toString()` de um array retorna algo como `[I@7852e922`, não os seus valores).

Semântica de Referências

Quando você declara uma variável de array, ela não armazena o array diretamente — ela armazena uma **referência** (um endereço de memória) para o objeto array que reside no **heap**. Essa é uma distinção crucial: ao atribuir `array2 = array1`, você não está criando uma cópia do array. Está fazendo com que `array2` aponte para o **mesmo objeto** na memória que `array1`. Qualquer modificação feita através de `array2` será imediatamente visível ao acessar `array1`, e vice-versa.

```
int[] a = {1, 2, 3};

int[] b = a; // b aponta para o MESMO array que a

b[0] = 99;

System.out.println(a[0]); // imprime 99 – a também foi afetado!

// Para criar uma cópia INDEPENDENTE, use Arrays.copyOf:

import java.util.Arrays;

int[] c = Arrays.copyOf(a, a.length);
```

```
c[0] = 42;
```

```
System.out.println(a[0]); // ainda 99 – a não foi afetado
```

■■ **Atenção:** Arrays de objetos têm **dois níveis de indireção**: a variável aponta para o array, e cada elemento do array aponta para um objeto. Isso significa que copiar o array (shallow copy) copia apenas as referências, não os objetos em si.

CAPÍTULO 04

O Enhanced For Loop (For-each)

Sintaxe com dois pontos, quando usar e quando preferir o for tradicional

O que é o Enhanced For Loop?

Introduzido no Java 5, o **enhanced for loop** (também chamado de **for-each**) é uma forma simplificada de percorrer todos os elementos de um array ou de qualquer coleção que implemente a interface `Iterable`. Sua principal vantagem é eliminar a necessidade de gerenciar manualmente um índice, tornando o código mais limpo, mais legível e menos propenso a erros como o clássico **off-by-one** (usar `<= length` em vez de `< length`).

Sintaxe e Funcionamento

A sintaxe do for-each usa **dois pontos (:)** como separador, em contraste com o ponto e vírgula (;) do for tradicional. O loop declara uma variável local que recebe, a cada iteração, o valor do próximo elemento do array — do primeiro (índice 0) até o último (índice `length-1`), em ordem.

```
String[] fruits = {"Apple", "Banana", "Cherry", "Date"};

// For-each – limpo e direto

for (String fruit : fruits) {

    System.out.println(fruit);

}

// Saída: Apple / Banana / Cherry / Date

// Equivalente com for tradicional

for (int i = 0; i < fruits.length; i++) {
```

```
System.out.println(fruits[i]);

}
```

Quando NÃO usar o For-each

O for-each é ideal quando você precisa apenas **ler** os elementos do array, sem necessidade de saber em qual posição você está. No entanto, existem situações em que o for tradicional é necessário:

- **Modificar elementos:** a variável local do for-each é uma cópia do valor — modificá-la não altera o array original.
- **Precisar do índice:** se você precisa saber a posição do elemento atual (ex: imprimir "Elemento 3: valor"), use o for tradicional.
- **Percorrer ao contrário:** o for-each sempre vai do início ao fim; para percorrer de trás para frente, use um for com decremento.
- **Iterar sobre dois arrays simultaneamente:** o for-each não permite controlar dois índices ao mesmo tempo.

```
int[] nums = {10, 20, 30};

// ERRADO: não modifica o array

for (int n : nums) {

    n = n * 2; // modifica apenas a cópia local

}

System.out.println(nums[0]); // ainda 10

// CORRETO: use o for tradicional para modificar

for (int i = 0; i < nums.length; i++) {

    nums[i] = nums[i] * 2;
```

```
}
```

```
System.out.println(nums[0]); // agora 20
```

CAPÍTULO 05

A Classe `java.util.Arrays`

Métodos utilitários: `sort`, `fill`, `copyOf`, `toString` e `equals`

Visão Geral

A classe `java.util.Arrays` é uma classe utilitária que fornece métodos estáticos para as operações mais comuns com arrays em Java. Ela é parte da biblioteca padrão e está disponível desde o Java 1.2. Para usá-la, basta adicionar `import java.util.Arrays;` no topo do seu arquivo. Nunca instancie essa classe — todos os métodos são estáticos e invocados diretamente pela classe.

`Arrays.sort` — Ordenação

`Arrays.sort(array)` ordena o array em **ordem crescente**, modificando o array original (operação in-place). Para tipos primitivos, usa uma variante do Quicksort otimizada (Dual-Pivot Quicksort). Para objetos, usa Timsort. Em ambos os casos, a complexidade é **$O(n \log n)$** no caso médio, o que é muito eficiente para a maioria dos cenários práticos.

`Arrays.fill` — Preenchimento

`Arrays.fill(array, valor)` preenche todos os elementos do array com o valor especificado. Também existe uma versão com faixas: `Arrays.fill(array, fromIndex, toIndex, valor)`, que preenche apenas o intervalo `[fromIndex, toIndex)` (inclusive/exclusivo).

`Arrays.copyOf` — Cópia

`Arrays.copyOf(array, novoTamanho)` retorna um **novo array** com os elementos copiados e o tamanho especificado. Se `novoTamanho` for maior que o array original, os elementos extras são preenchidos com o valor padrão do tipo. Se for menor, os elementos excedentes são descartados. Útil para redimensionar arrays de forma controlada.

`Arrays.toString` — Representação Textual

`Arrays.toString(array)` retorna uma representação legível do array como String, no formato `[v1, v2, v3, ...]`. Sem esse método, imprimir o array diretamente com `System.out.println(array)` resultaria em algo como `I@7852e922` — que é o endereço de memória do objeto, não os seus valores.

```
import java.util.Arrays;

int[] nums = {5, 3, 8, 1, 9, 2};

System.out.println(Arrays.toString(nums)); // [5, 3, 8, 1, 9, 2]

Arrays.sort(nums);

System.out.println(Arrays.toString(nums)); // [1, 2, 3, 5, 8, 9]

Arrays.fill(nums, 0);

System.out.println(Arrays.toString(nums)); // [0, 0, 0, 0, 0, 0]

int[] copy = Arrays.copyOf(nums, 10);

System.out.println(Arrays.toString(copy)); // [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

// Arrays.equals – compara elemento a elemento (diferente de ==)

int[] a = {1, 2, 3};

int[] b = {1, 2, 3};

System.out.println(a == b); // false (referências diferentes)

System.out.println(Arrays.equals(a,b)); // true (mesmos valores)
```

CAPÍTULO 06

Busca em Arrays

Busca sequencial vs busca binária — complexidade e `Arrays.binarySearch`

Busca Linear (Sequencial)

A busca linear é a forma mais simples de encontrar um elemento em um array: você percorre cada elemento, do início ao fim, comparando-o com o valor procurado. Sua complexidade é **$O(n)$** — no pior caso, você examina todos os N elementos antes de encontrar o que procura (ou concluir que ele não existe). Para arrays pequenos ou não ordenados, a busca linear é suficiente e até preferível pela sua simplicidade.

```
// Busca linear – funciona em arrays não ordenados

static int linearSearch(int[] arr, int target) {

    for (int i = 0; i < arr.length; i++) {

        if (arr[i] == target) return i; // retorna o índice encontrado

    }

    return -1; // não encontrado

}
```

Busca Binária

A busca binária é um algoritmo muito mais eficiente, com complexidade **$O(\log n)$** . A ideia é simples: em um array ordenado, compare o valor procurado com o elemento do meio. Se for igual, encontrou. Se for menor, busque na metade esquerda. Se for maior, busque na metade direita. A cada iteração, o espaço de busca é reduzido à metade — um array com 1 bilhão de elementos requer no máximo 30 comparações!

O método `Arrays.binarySearch(array, valor)` implementa essa lógica para você. Ele retorna o **índice** do elemento encontrado, ou um **valor negativo** se o elemento não existir no array. O valor negativo retornado é **$-(\text{ponto_de_inserção}) - 1$** , o que pode ser útil para

saber onde o elemento deveria ser inserido para manter a ordenação.

```
import java.util.Arrays;

int[] sorted = {2, 5, 10, 15, 20, 25, 30};

// 0 1 2 3 4 5 6

int idx = Arrays.binarySearch(sorted, 15); // retorna 3

int miss = Arrays.binarySearch(sorted, 7); // retorna negativo (-3)

System.out.println("Índice de 15: " + idx); // 3

System.out.println("7 encontrado? " + (miss >= 0)); // false
```

■■ **Atenção: REQUISITO OBRIGATÓRIO:** o array DEVE estar ordenado em ordem crescente antes de chamar `Arrays.binarySearch`. Usar o método em um array não ordenado produz resultados imprevisíveis. Se necessário, chame `Arrays.sort(array)` antes.

CAPÍTULO 07

Variable Arguments (Varargs)

A sintaxe ... que permite métodos com número variável de argumentos

O que são Varargs?

Varargs (abreviação de *variable arguments*) é um recurso introduzido no Java 5 que permite declarar métodos capazes de receber um número variável de argumentos — de zero até qualquer quantidade. Por baixo dos panos, o Java trata os varargs como um array: você pode iterar sobre eles, verificar `args.length` e acessar elementos com índices normalmente.

Sintaxe e Regras

A sintaxe usa **três pontos (...)** após o tipo do parâmetro. Por exemplo, `void metodo(int... nums)` permite chamar `metodo()`, `metodo(1)`, `metodo(1, 2, 3)` ou até passar um array diretamente: `metodo(new int[]{1, 2, 3})`.

- Só pode haver **UM parâmetro vararg** por método.
- O parâmetro vararg DEVE ser o **último** na lista de parâmetros.
- É possível combinar parâmetros normais antes do vararg: `void log(String prefix, int... vals)`.

```
// Sem varargs – chamada obrigatoriamente com array

static void printAll(String[] args) {

    for (String s : args) System.out.println(s);

}

printAll(new String[]{"A", "B", "C"}); // verboso


// Com varargs – chamada direta

static void printAll(String... args) {
```



```
for (String s : args) System.out.println(s);

}

printAll("A", "B", "C"); // simples

printAll(); // zero argumentos: OK

printAll(new String[]{"X", "Y"}); // ainda aceita array


// Combinando parâmetros normais com varargs

static double sum(double first, double... rest) {

    double total = first;

    for (double v : rest) total += v;

    return total;

}

System.out.println(sum(1.0, 2.0, 3.0, 4.0)); // 10.0
```

CAPÍTULO 08

Arrays Bidimensionais

Matrizes, índices de linha e coluna, inicialização e jagged arrays

O que é um Array 2D?

Um array bidimensional (2D) é, na prática, um **array de arrays**. Você pode visualizá-lo como uma tabela com linhas e colunas — semelhante a uma planilha. Em Java, declara-se com `int[][]`, onde o primeiro par de colchetes representa as linhas e o segundo representa as colunas. Arrays 2D são amplamente usados para representar matrizes matemáticas, tabuleiros de jogos, imagens em pixels e dados tabulares em geral.

Declaração, Inicialização e Acesso

Para acessar um elemento, usa-se dois índices: `matrix[linha][coluna]`. Para percorrer todos os elementos, utiliza-se dois loops aninhados — um para as linhas e outro para as colunas. O for-each aninhado também funciona perfeitamente com arrays 2D.

```
// Declaração com new

int[][] grid = new int[3][4]; // 3 linhas, 4 colunas — tudo zerado

// Declaração com initializer

int[][] matrix = {

    {1, 2, 3}, // linha 0

    {4, 5, 6}, // linha 1

    {7, 8, 9} // linha 2

};
```

```
System.out.println(matrix[1][2]); // 6 (linha 1, coluna 2)

System.out.println(matrix.length); // 3 (número de linhas)

System.out.println(matrix[0].length); // 3 (colunas da linha 0)


// Percorrendo com for-each aninhado

for (int[] row : matrix) {

    for (int val : row) {

        System.out.print(val + " ");

    }

    System.out.println();

}
```

Jagged Arrays — Arrays com Linhas de Tamanhos Diferentes

Em Java, os arrays internos de um array 2D podem ter tamanhos diferentes — isso é chamado de **jagged array** (array irregular). Isso é possível porque, internamente, cada linha é um array separado e independente. Esse recurso pode ser útil para representar estruturas triangulares ou qualquer tabela onde as linhas têm comprimentos variáveis.

```
// Jagged array – linhas de tamanhos diferentes

int[][] jagged = new int[3][];

jagged[0] = new int[]{1};

jagged[1] = new int[]{2, 3};

jagged[2] = new int[]{4, 5, 6};


// Percorrer sem assumir tamanho fixo
```

```
for (int[] row : jagged) {  
  
    System.out.println(Arrays.toString(row));  
  
}  
  
// [1]  
  
// [2, 3]  
  
// [4, 5, 6]
```

CAPÍTULO 09

Arrays Multidimensionais

Além do 2D — arrays 3D, 4D e casos de uso práticos

Arrays com mais de Duas Dimensões

Java suporta arrays com qualquer número de dimensões. Um array 3D, por exemplo, pode ser declarado como `int[][][]` e visualizado como um cubo de valores. Na prática, arrays com mais de 3 dimensões são raros — a complexidade de gerenciamento tende a superar os benefícios, e estruturas de dados mais ricas (como listas de listas ou classes personalizadas) são preferidas.

Os casos de uso mais comuns para arrays 3D incluem: representação de imagens em cores (largura × altura × canais RGB), simulações físicas tridimensionais, e problemas de programação dinâmica com três estados variáveis.

```
// Array 3D: "profundidade" x linhas x colunas

int[][][] cube = new int[2][3][4]; // 2 "fatias" de 3x4

// Acessar um elemento

cube[0][1][2] = 42;

System.out.println(cube[0][1][2]); // 42

// Arrays.deepToString para representar arrays multidimensionais

import java.util.Arrays;

int[][] mat = {{1,2},{3,4}};

System.out.println(Arrays.deepToString(mat)); // [[1, 2], [3, 4]]
```

■ **Dica:** Para imprimir arrays multidimensionais de forma legível, use `Arrays.deepToString(array)` em vez de `Arrays.toString(array)`. O segundo apenas imprime os endereços de memória dos arrays internos.

CAPÍTULO 10

Desafio: Elemento Mínimo

Ler inteiros do console e encontrar o menor valor com findMin

Descrição do Problema

Neste desafio, você deve implementar dois métodos: `readIntegers` e `findMin`. O método `readIntegers` deve ler uma linha do console contendo números inteiros separados por vírgula e retornar um array `int[]` com esses valores. Já o método `findMin` deve receber esse array e retornar o menor elemento encontrado.

Estratégia de Implementação

Para `readIntegers`: leia a linha com `Scanner.nextLine()`, divida-a pelo separador vírgula com `String.split(",")`, converta cada elemento para inteiro com `Integer.parseInt()` e preencha o array. Para `findMin`: inicialize o mínimo com o primeiro elemento e percorra o restante do array — se encontrar um valor menor, atualize o mínimo.

```
import java.util.Arrays;

import java.util.Scanner;

static int[] readIntegers(Scanner scanner) {

    String line = scanner.nextLine();

    String[] parts = line.split(",");

    int[] result = new int[parts.length];

    for (int i = 0; i < parts.length; i++) {

        result[i] = Integer.parseInt(parts[i].trim());

    }
```

```
return result;

}

static int findMin(int[] array) {

    int min = array[0];

    for (int val : array) {

        if (val < min) min = val;

    }

    return min;

}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    int[] nums = readIntegers(sc);

    System.out.println("Array: " + Arrays.toString(nums));

    System.out.println("Mínimo: " + findMin(nums));

}

// Entrada: 7,3,19,4,1,8

// Saída: Array: [7, 3, 19, 4, 1, 8] / Mínimo: 1
```


CAPÍTULO 11

Desafio: Inversão de Array

Trocar elementos simetricamente até o meio, sem criar um novo array

Descrição do Problema

Implemente um método `reverse(int[] array)` que inverta os elementos do array **in-place** — sem criar um novo array auxiliar. Imprima o array antes e depois da operação usando `Arrays.toString`.

Estratégia: Troca Simétrica até o Meio

A ideia central é simples e elegante: troque o elemento na posição 0 com o elemento na última posição, depois o elemento na posição 1 com o penúltimo, e assim por diante. O loop continua até o **meio** do array — se o array tiver comprimento ímpar, o elemento central nunca precisa ser movido. Para a troca, usa-se uma variável auxiliar temporária `temp`.

```
static void reverse(int[] arr) {

    for (int i = 0; i < arr.length / 2; i++) {

        int temp = arr[i];

        arr[i] = arr[arr.length - 1 - i];

        arr[arr.length-1-i] = temp;

    }

}

public static void main(String[] args) {

    int[] nums = {1, 2, 3, 4, 5, 6};
```

```
System.out.println("Antes: " + Arrays.toString(nums));

reverse(nums);

System.out.println("Depois: " + Arrays.toString(nums));

}

// Antes: [1, 2, 3, 4, 5, 6]

// Depois: [6, 5, 4, 3, 2, 1]
```

■ **Regra:** Como o método recebe uma referência ao array, as modificações feitas dentro de `reverse` afetam diretamente o array original — não é necessário retornar nada.

CAPÍTULO 12

Desafio: Ordenação Decrescente

Gerar inteiros aleatórios e ordenar do maior para o menor com Comparator

Descrição do Problema

Crie um array de 10 inteiros gerados aleatoriamente com `Random.nextInt`. Imprima o array antes de ordenar, depois ordene em **ordem decrescente** (do maior para o menor) e imprima novamente.

O Desafio: `Comparator.reverseOrder()`

O método `Arrays.sort` possui uma sobrecarga que aceita um `Comparator`: `Arrays.sort(array, Comparator)`. Usando `Comparator.reverseOrder()`, a ordenação é feita em ordem decrescente automaticamente. **Porém, há uma pegadinha importante:** `Comparator.reverseOrder()` só funciona com **tipos de referência** como `Integer`, `Double`, etc. — **não funciona com tipos primitivos** como `int` ou `double`. Por isso, usamos `Integer[]` em vez de `int[]`.

```
import java.util.Arrays;

import java.util.Comparator;

import java.util.Random;

public static void main(String[] args) {

    Integer[] nums = new Integer[10]; // Integer[], nao int[]

    Random rand = new Random();

    for (int i = 0; i < nums.length; i++) {

        nums[i] = rand.nextInt(100); // valores de 0 a 99
    }
}
```

```
}

System.out.println("Antes: " + Arrays.toString(nums));

Arrays.sort(nums, Comparator.reverseOrder());

System.out.println("Depois: " + Arrays.toString(nums));

}

// Exemplo de saída:

// Antes: [34, 72, 5, 88, 21, 66, 44, 91, 13, 57]

// Depois: [91, 88, 72, 66, 57, 44, 34, 21, 13, 5]
```

■■ **Atenção:** `Comparator.reverseOrder()` exige tipos de referência (`Integer`, `Double`...). Usar com `int[]` causa erro de compilação. Autoboxing resolve: declare o array como `Integer[]`.

Parabéns por concluir a apostila!

Você explorou desde a declaração básica de arrays até arrays multidimensionais, passando por varargs, busca binária e três desafios práticos. Arrays são a base de inúmeras estruturas de dados avançadas — dominar esse fundamento abre as portas para ArrayList, HashMap, árvores e muito mais.

Java Arrays — Complete Java Masterclass